Product and Category Management System

Week 2 Task — DBMS

Name: Yokesh

Class: B.Sc CS (AI & ML)

Roll Number: 6253210211062

---

1. Objective

To design a database system for managing Products and Categories, establish primary and foreign key relationships between them, and perform insertion, updating, deletion, and reporting operations using SQL.

2. Entity Design

Two entities are used in this system:

Entity	Description	Key Attributes
Category	Stores product categories (parent entity)	category_id (PK), category_name, description
Product	Stores product details (child entity)	product_id (PK), product_name, category_id (FK), price, stock

3. Primary and Foreign Key Relationship

The Category table is the parent entity with category_id as its Primary Key. The Product table is the child entity with product_id as its Primary Key and category_id as a Foreign Key referencing Category(category_id). This establishes a one-to-many relationship: one category can have many products. ON DELETE CASCADE ensures that deleting a category removes its associated products.

4. SQL Implementation

The complete SQL script:

```
-- ============================================================
-- PRODUCT AND CATEGORY MANAGEMENT SYSTEM
-- ============================================================

-- 1. CREATE DATABASE
CREATE DATABASE ProductCategoryDB;
USE ProductCategoryDB;

-- ============================================================
-- 2. TABLE DESIGN WITH PRIMARY & FOREIGN KEY RELATIONSHIPS
-- ============================================================

-- Category Table (Parent)
```

```
CREATE TABLE Category (
    category_id   INT PRIMARY KEY AUTO_INCREMENT,
    category_name VARCHAR(50) NOT NULL UNIQUE,
    description   VARCHAR(200)
);

-- Product Table (Child) -- references Category
CREATE TABLE Product (
    product_id   INT PRIMARY KEY AUTO_INCREMENT,
    product_name VARCHAR(100) NOT NULL,
    category_id  INT NOT NULL,
    price        DECIMAL(10,2) NOT NULL CHECK (price >= 0),
    stock        INT NOT NULL DEFAULT 0 CHECK (stock >= 0),
    FOREIGN KEY (category_id) REFERENCES Category(category_id)
        ON DELETE CASCADE
        ON UPDATE CASCADE
);

-- ============================================================
-- 3. INSERTING SAMPLE DATA
-- ============================================================

INSERT INTO Category (category_name, description) VALUES
('Electronics', 'Electronic gadgets and devices'),
('Groceries', 'Daily food and household items'),
('Stationery', 'Office and school supplies');

INSERT INTO Product (product_name, category_id, price, stock) VALUES
('Laptop', 1, 55000.00, 10),
('Smartphone', 1, 20000.00, 25),
('Rice Bag (5kg)', 2, 450.00, 100),
('Sugar (1kg)', 2, 45.00, 200),
('Notebook', 3, 30.00, 150),
('Pen Pack', 3, 20.00, 300);

-- ============================================================
-- 4. UPDATE OPERATIONS
-- ============================================================

-- Update price of a product
UPDATE Product
SET price = 52000.00
WHERE product_name = 'Laptop';

-- Update stock after a sale
UPDATE Product
SET stock = stock - 5
WHERE product_id = 2;

-- ============================================================
-- 5. DELETE OPERATIONS
-- ============================================================

-- Delete a specific product
DELETE FROM Product
WHERE product_name = 'Pen Pack';

-- Delete a category (also deletes its products due to ON DELETE CASCADE)
-- DELETE FROM Category WHERE category_id = 3;

-- ============================================================
-- 6. CATEGORY-WISE PRODUCT REPORTS
-- ============================================================

-- a) List all products along with their category name
```

```
SELECT p.product_id, p.product_name, c.category_name, p.price, p.stock
FROM Product p
JOIN Category c ON p.category_id = c.category_id
ORDER BY c.category_name;

-- b) Total number of products per category
SELECT c.category_name, COUNT(p.product_id) AS total_products
FROM Category c
LEFT JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name;

-- c) Total stock value per category (price * stock)
SELECT c.category_name, SUM(p.price * p.stock) AS total_stock_value
FROM Category c
JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name;

-- d) Category with the highest number of products
SELECT c.category_name, COUNT(p.product_id) AS total_products
FROM Category c
JOIN Product p ON c.category_id = p.category_id
GROUP BY c.category_name
ORDER BY total_products DESC
LIMIT 1;

-- e) Low stock report (stock less than 20) grouped by category
SELECT c.category_name, p.product_name, p.stock
FROM Product p
JOIN Category c ON p.category_id = c.category_id
WHERE p.stock < 20
ORDER BY c.category_name;
```

5. Operations Performed

Operation	Description
Create	Created Category and Product tables with PK-FK relationship
Insert	Inserted sample categories (Electronics, Groceries, Stationery) and products
Update	Updated product price and reduced stock after a sale
Delete	Deleted a product record; category deletion cascades to its products
Report	Generated category-wise product listings, counts, stock value, and low-stock alerts

6. Category-wise Reports Generated

a) List of all products with their category name

b) Total number of products per category

c) Total stock value per category (price × stock)

d) Category with the highest number of products

e) Low-stock report (stock less than 20) grouped by category

7. Conclusion

This task demonstrates the design of a relational database for product and category management, using primary and foreign keys to maintain referential integrity, and SQL queries to perform CRUD operations and generate meaningful category-wise reports.